Shaker Lab

Мобильное приложение-справочник коктейлей

Кирилл Ковалев 3 курс 6 группа

Проблема и актульность

Почему приложение умеет смысл как пользовательский продукт

Проблема

Рецепты разбросаны по разным сайтам. Пользователю неудобно искать состав, сохранять рецепты и понимать, что можно приготовить из игредиентов дома.

Решение

Приложение собирает каталог, поиск, избранное и домашний бар в одном месте.

Что получает пользователь

Быстрый поиск
подробные рецепты
Подбор по игредиентам
Синхронизация

Цель и задачи курсовой работы

Что нужно было спроектировать и реализовать.

Цель: разработать Android-приложение для поиска, просмотра и сохранения рецептов коктейлей, а также подбора напитков по домашним ингредиентам.

- спроектировать структуру приложения и пользовательские сценарии
- разработать интерфейс и основные экраны
- подключить внешний источник данных TheCocktailDB API
- реализовать локальное хранение избранного и ингредиентов бара
- добавить Google Sign-In и синхронизацию с Firebase

Используемые технологии

Основные библиотеки и за что они отвечают.

Навигация	Navigation Component
Асинхронность	Coroutines
UI	XML + Material 3
Сеть	Retrofit + OkHttp
JSON	Gson
API	TheCocktailDB
База	Room / SQLite
История	SharedPreferences
Изображения	Glide
Auth	Firebase Auth
Облако	Cloud Firestore
DI	Koin 4.0

Архитектурный подход

Экран не работает напрямую с API или базой данных.

PRESENTATION

Fragment + ViewModel + LiveData

UI-слой отображает данные и реагирует на действия пользователя.

DOMAIN

16 Use Cases · модели · интерфейсы

Бизнес-логика не зависит от Android, Room, Firebase и Retrofit.

DATA

API · Room · Firestore · SharedPreferences

Реальные источники данных и реализации репозиториев.

Что даёт?

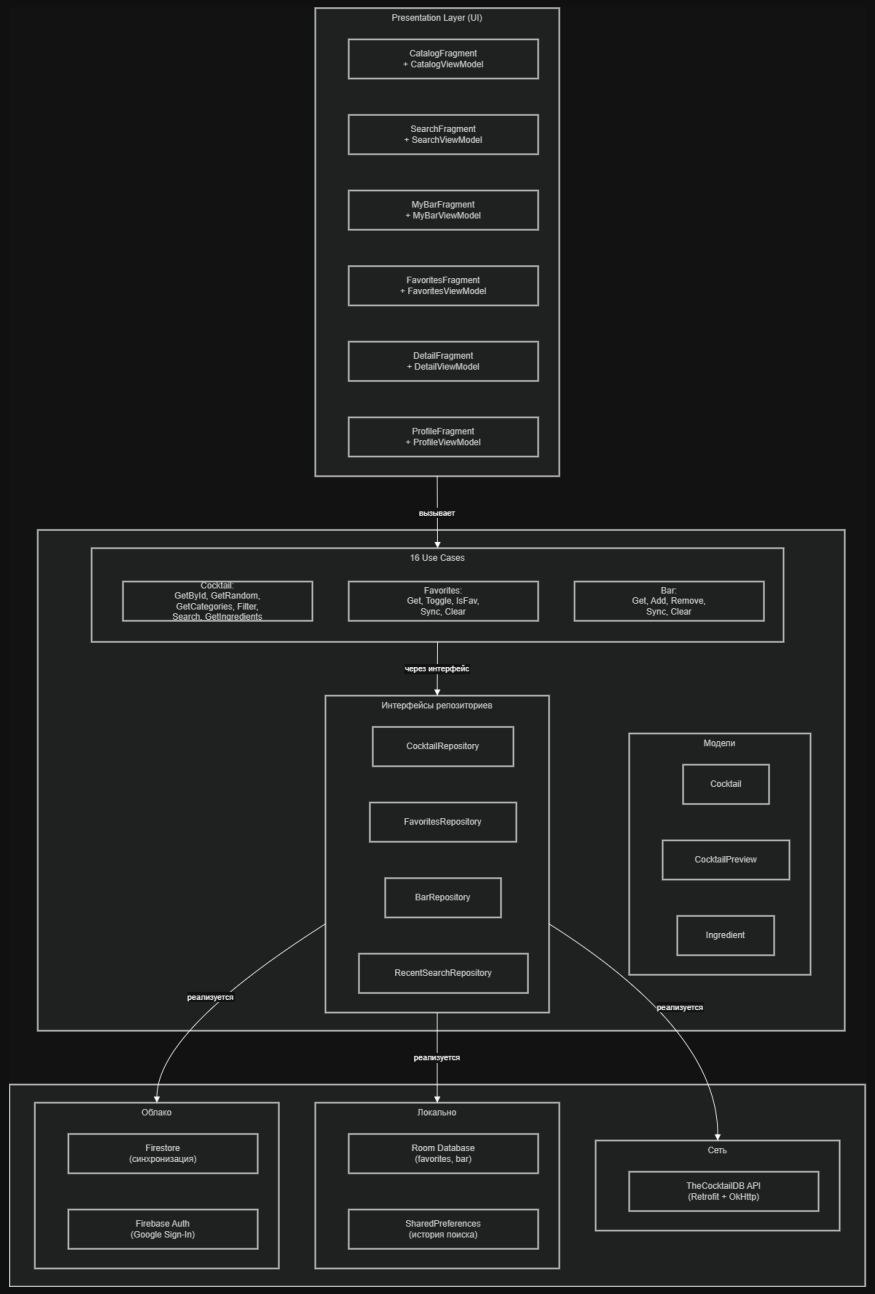
- каждый класс отвечает за одно
- легче читать и расширять
- ViewModel не знает про API и Room

Структура архитектуры проекта

Как слои, репозитории, модели и источники данных связаны между собой.

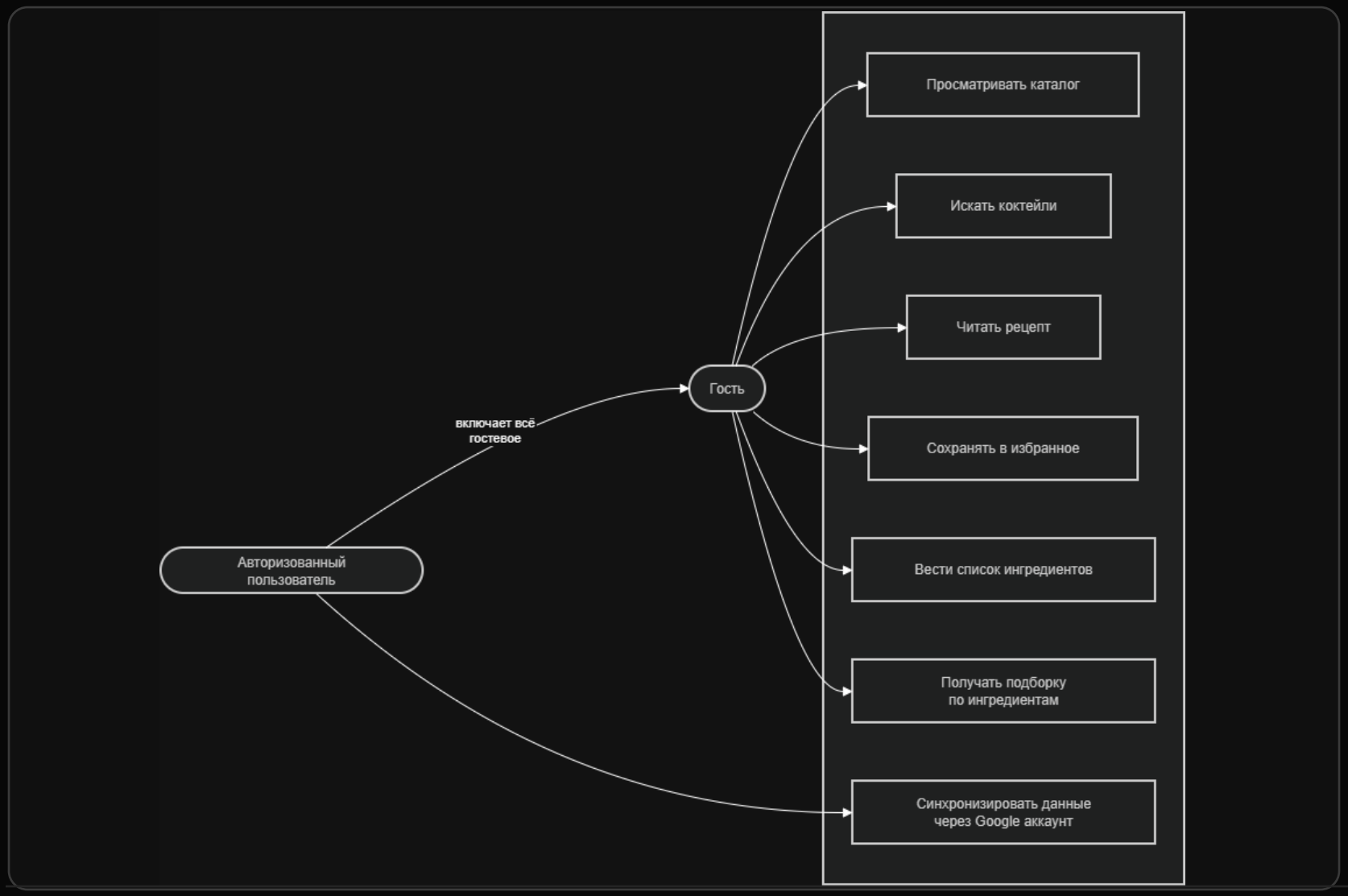
Ключевая идея

- **UI:** Fragment + ViewModel
- **Use Cases:** единая бизнес-логика
- **Repositories:** контракты в domain
- **Data:** Room, Firestore, API



Пользовательские сценарии системы

Диаграмма показывает действия пользователя, а не внутренний код.

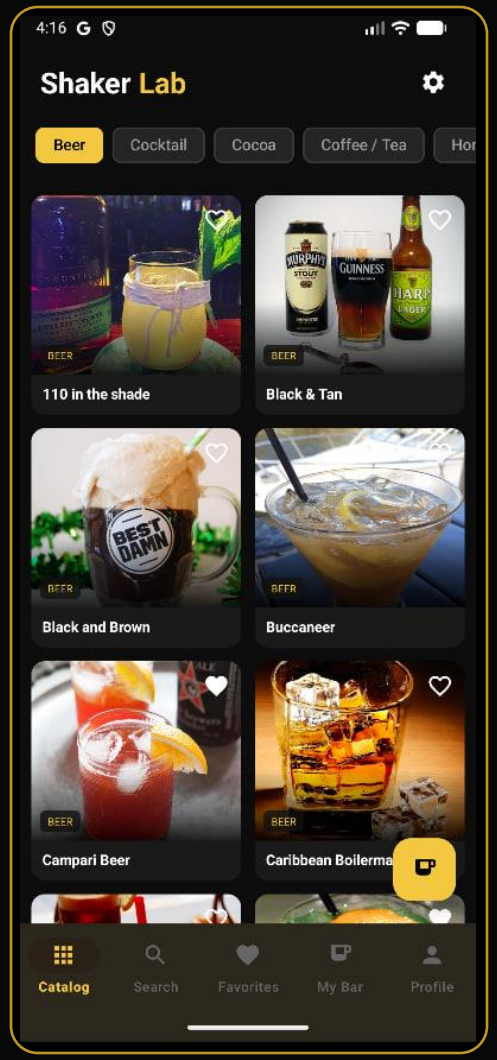


Смысл диаграммы

- **Гость:** каталог, поиск, рецепты, избранное, бар
- **Авторизованный:** все то же, но с синхронизация
- **Внешние сервисы:** API рецептов и Firebase

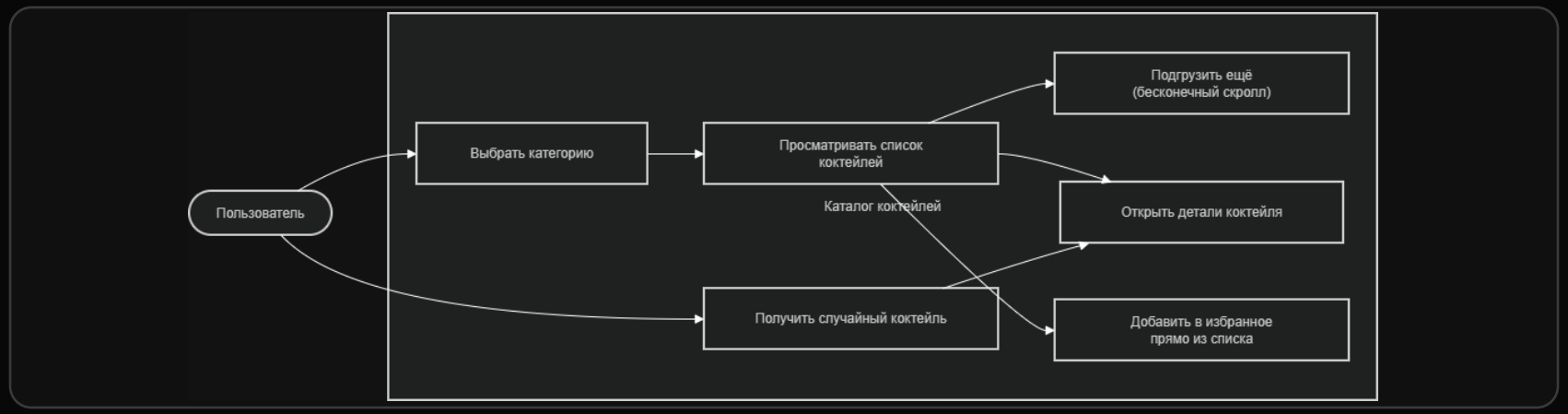
Функциональность каталога

Первый экран: категории, карточки, избранное и случайный рецепт.



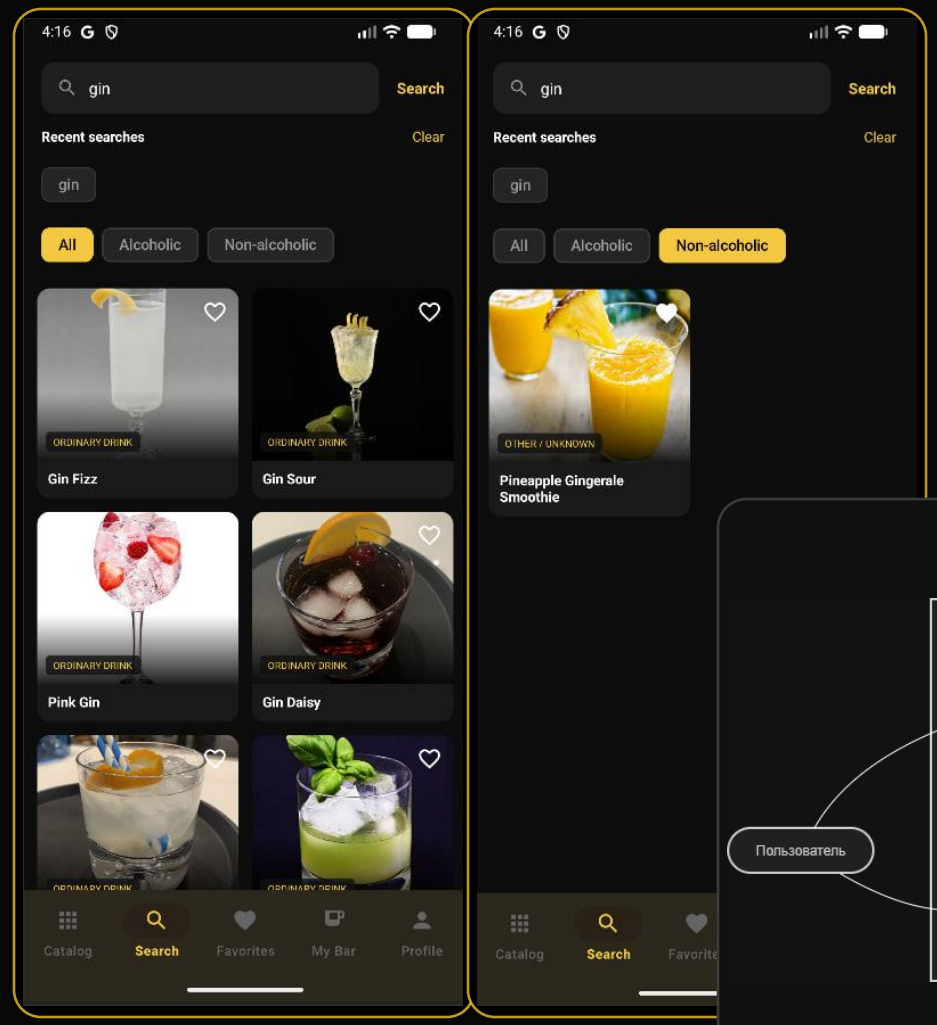
Что умеет экран

- фильтрация по категории
- бесконечный скролл
- кнопка «Случайный»
- избранное прямо из карточки

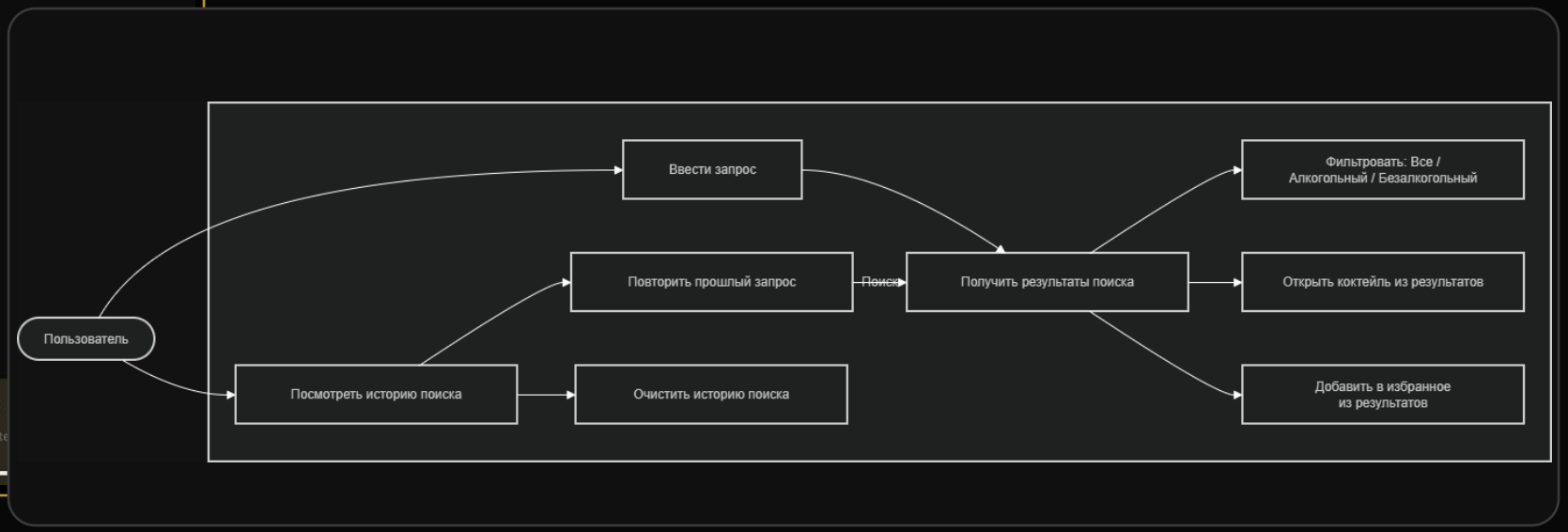


Функциональность поиска

Запрос, история последних поисков и фильтрация по типу напитка.

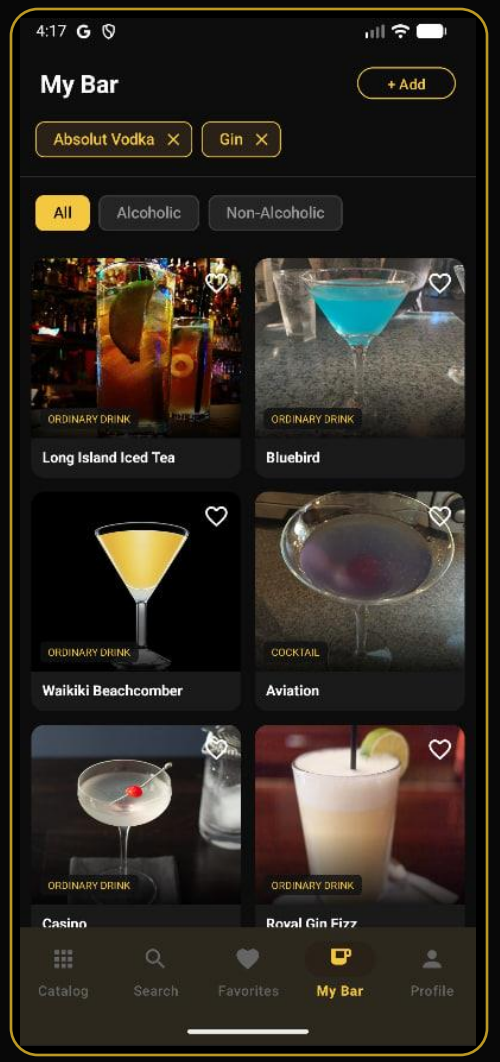


- поиск по названию через API
- история последних 8 запросов
- фильтр: все / алкогольный / безалкогольный
- загрузка, пустой результат, ошибка



Подбор коктейлей по ингредиентам

Подбор коктейлей по ингредиентам, которые уже есть дома.

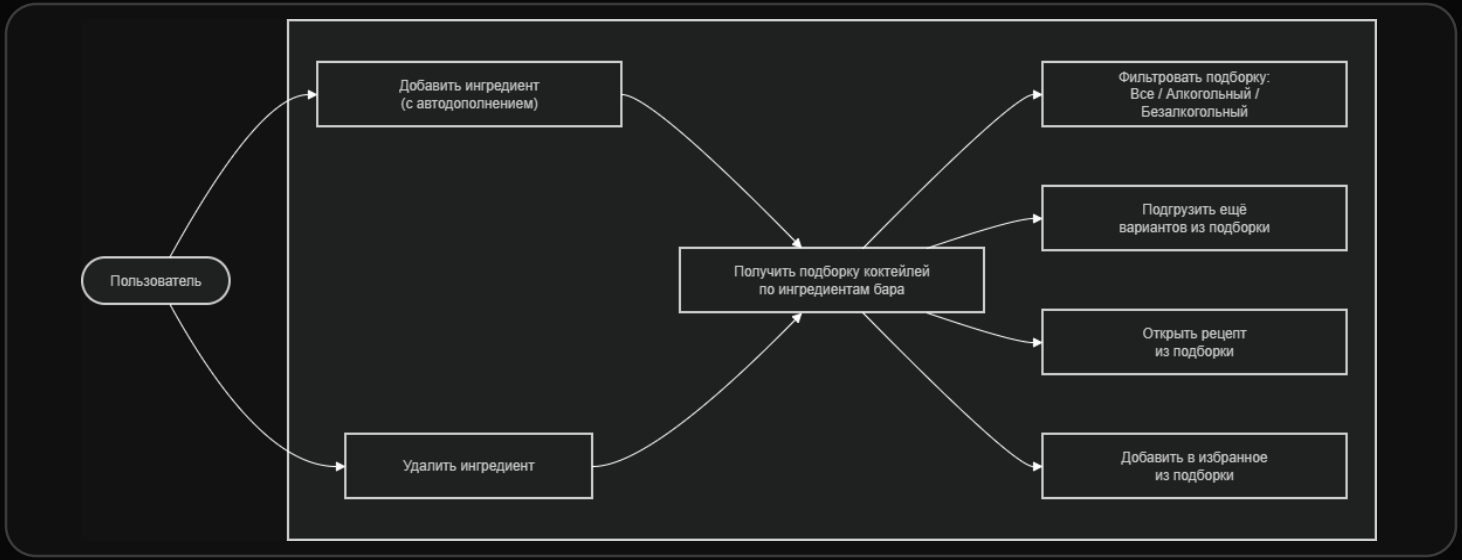


Как работает

- добавить ингредиенты
- автодополнение
- подборка коктейлей
- фильтр по алкоголю
- бесконечная прокрутка

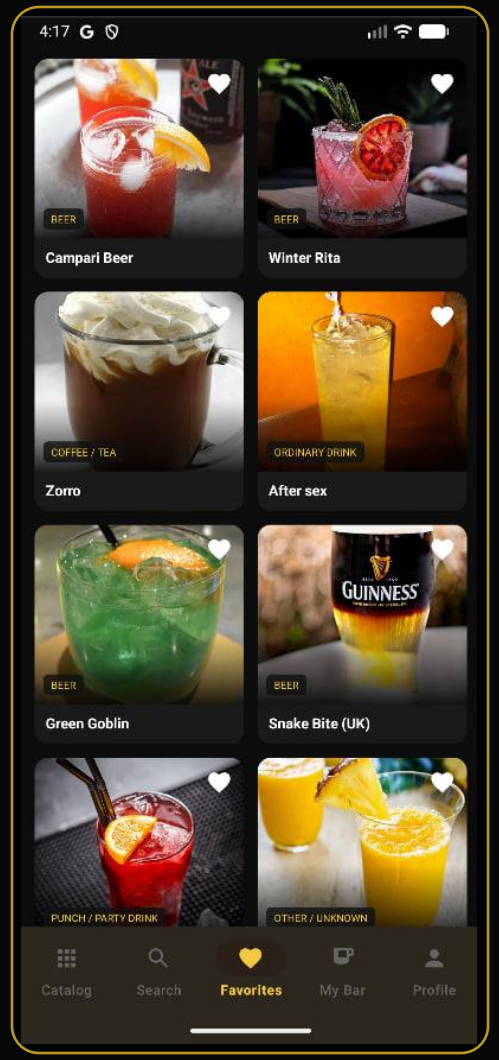
Техническая деталь

Бесплатный API ограничен: нет удобного списка с полным составом. Решение — 40 параллельных random-запросов и фильтрация на клиенте.

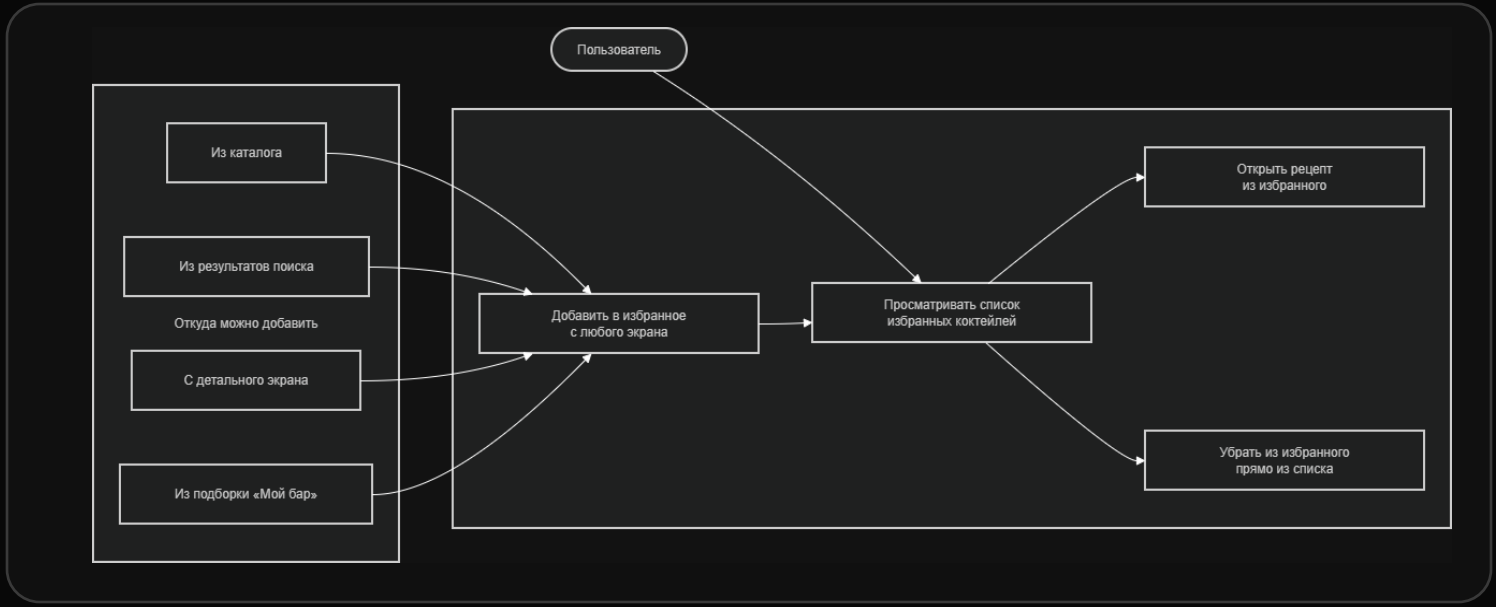


Локальное хранение избранного

Сохранённые коктейли доступны локально и обновляются реактивно.



- хранится локально в Room
- работает после перезапуска
- добавить можно с любого экрана
- удаление прямо из списка
- реактивное обновление через LiveData

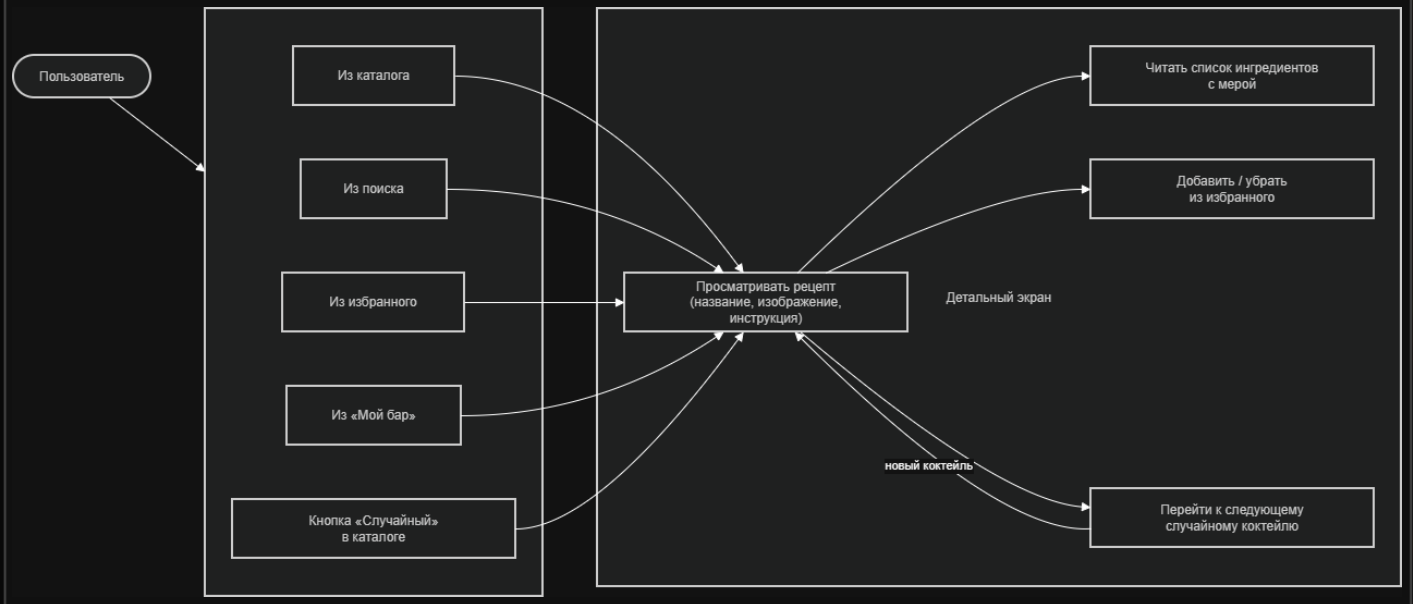


Экран подробного рецепта

Единый детальный экран, куда можно попасть из разных сценариев.

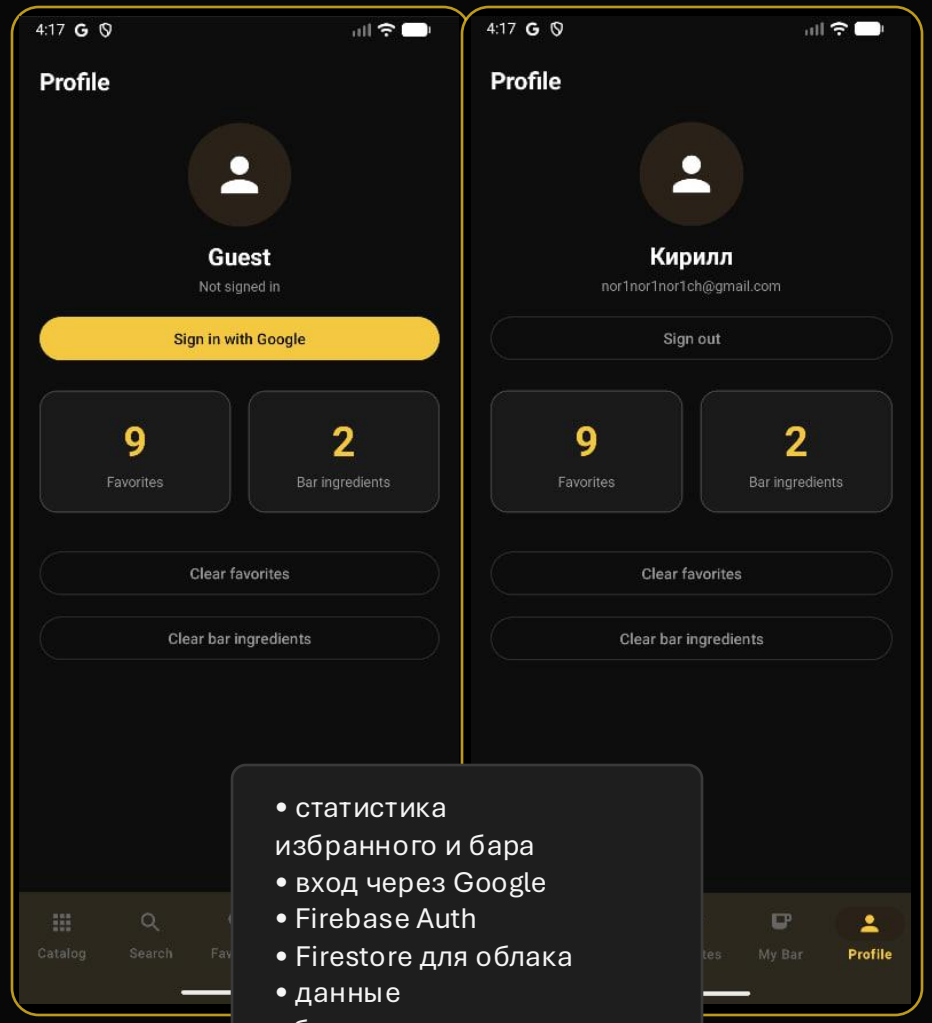
На экране рецепта

- крупное изображение и название
- категория и тип напитка
- ингредиенты с мерами
- инструкция приготовления
- кнопка избранного
- следующий случайный коктейль

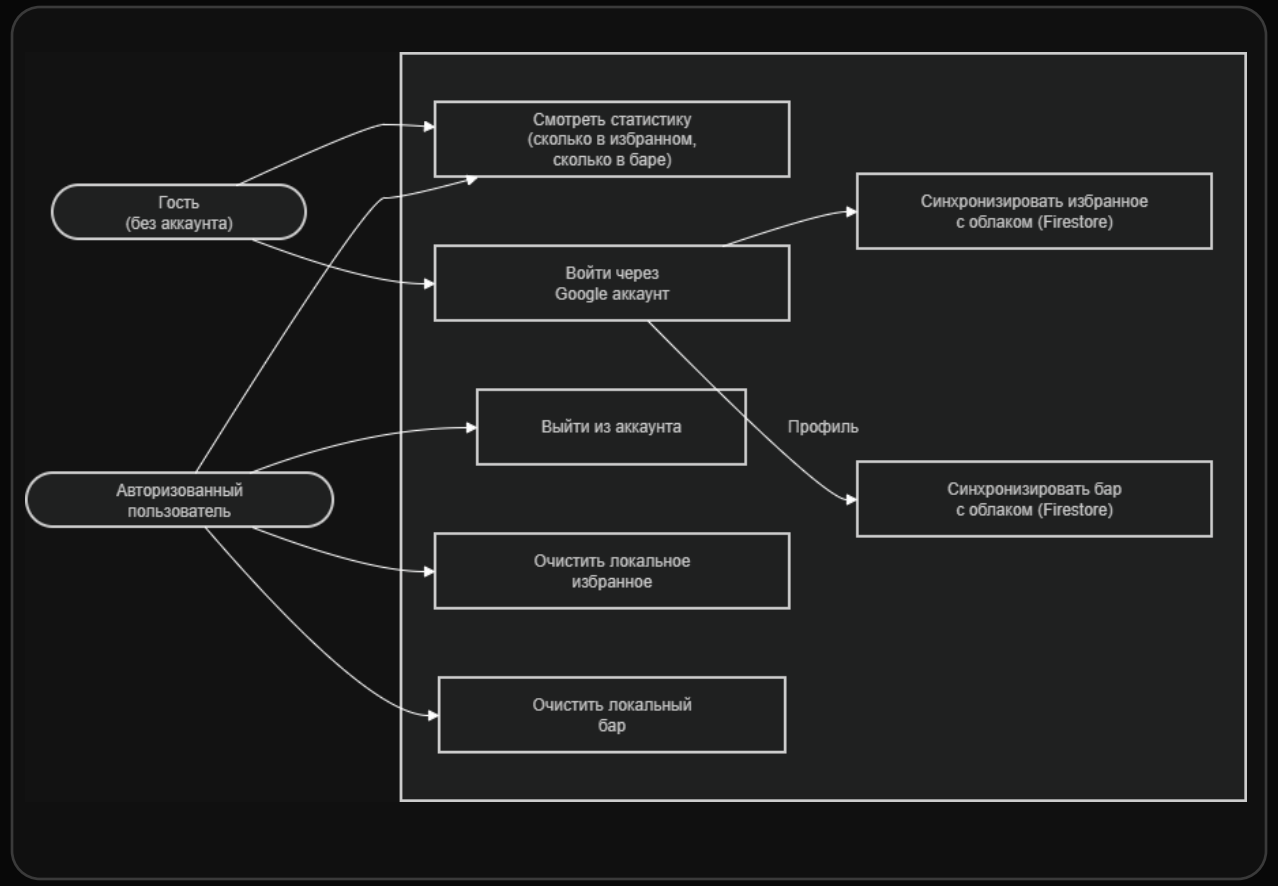


Профиль пользователя и облачная синхронизация

Без аккаунта — локально; после входа — синхронизация через Firebase.



- статистика избранного и бара
- вход через Google
- Firebase Auth
- Firestore для облака
- данные объединяются, а не затираются



Источники данных и интеграции

Сеть, локальное хранение и облако работают через Repository.

Room даёт офлайн-доступ к избранному и бару. Firebase используется после входа через Google.

Основной пользовательский сценарий

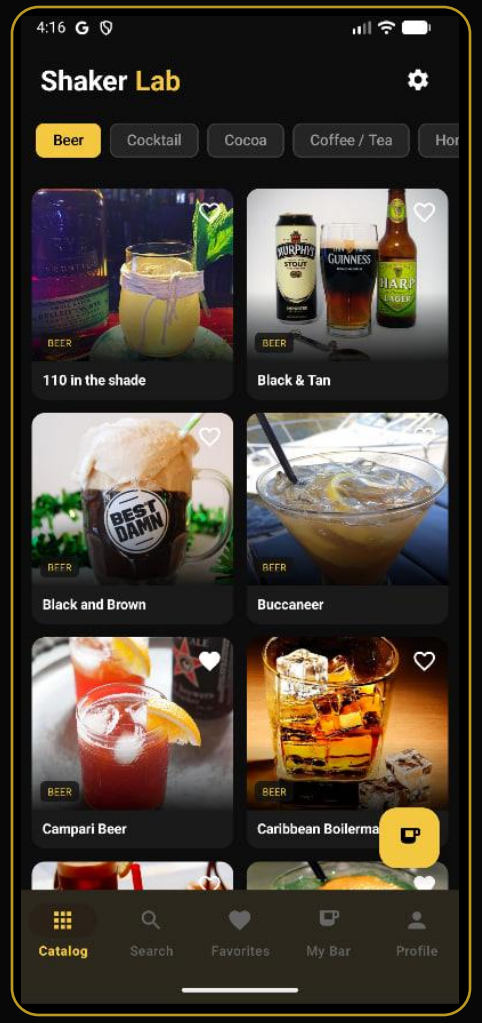
Как пользователь проходит приложение от запуска до приготовления коктейля.

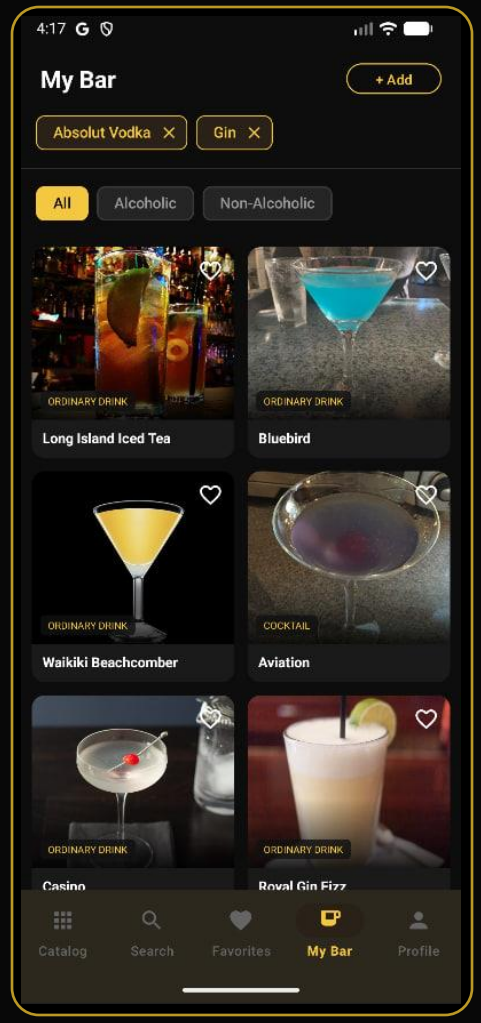
Открыл приложение → Просмотрел каталог → Нашёл рецепт → Добавил в избранное → Зашёл в «Мой бар» → Добавил ингредиенты → Получил подборку → **Приготовил коктейль**

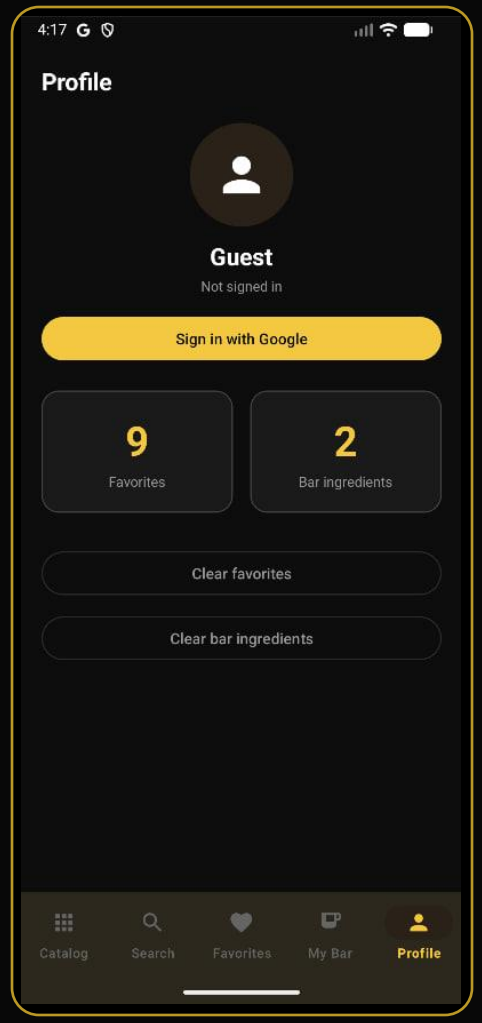
Сценарий показывает главную ценность приложения: не просто посмотреть рецепт, а понять, что реально можно приготовить из того, что есть дома.

Сценарий демонстрации приложения

Короткий сценарий живого показа на устройстве.







Результаты реализации и технические сложности

Итог реализации и реальные технические проблемы.

Что получилось

- работающее приложение с 6 экранами
- реальный API с живыми данными
- локальное хранение через Room
- облачная синхронизация Firebase
- бесконечный скролл и параллельные запросы

Сложности

- **Ограничение API:** обход через 40 getRandom()
- **Infinite scroll:** начальная загрузка минимум 10
- **Архитектура:** разделение Context, ViewModel и репозиториев
- **Реактивность избранного:** MediatorLiveData и Room

Перспективы развития проекта

Проект можно развивать как полноценный продукт.

Платная версия API

нормальная фильтрация по ингредиентам без обходного решения

Офлайн-режим

кэш ответов API через Room или OkHttp Cache

Анимации переходов

Shared Element Transition для карточки рецепта

Shaker Lab

Мобильное приложение-справочник коктейлей

Кирилл Ковалев 3 курс 6 группа